技术文档（完整版）

> 本文件覆盖项目背景、技术选型、系统架构、核心算法机制（AI端）、后端实现、测试与部署。可直接导出为 PDF 提交。

1. 项目背景与需求概述

1.1 项目背景与痛点

在高校与职场中，面对冗长的公文、比赛简章或审批通知，用户往往需要耗费大量时间人工拆解“需要交什么材料”、“何时截止”以及“具体办理流程”。当前的通用大模型（LLM）虽然具备优秀的阅读能力，但在直接应用于此类场景时存在明显缺陷：

1. **信息碎片化**：要求分散在长文各处，人工核对极易漏看。
2. **模型幻觉**：通用大模型容易捏造原文中不存在的规定或要求，导致用户准备错误。
3. **缺乏结构化交互**：纯文本答复无法直接转化为可勾选、可推进的“办事清单”。

1.2 核心目标与用户场景

本项目旨在打造一款面向学生、社团内勤、职场人士的**“智能办事清单助手（不漏办）”**。系统通过“文本/图片/PDF输入 → AI解析 → 结构化清单生成 → 状态动态联动”的闭环，实现：

- **精准提取**：将长篇通知转化为结构化的 JSON 清单。
- **防幻觉溯源**：每一条要求必须绑定原文，没有依据的标为“系统建议”。
- **动态缺项核对**：根据用户勾选，实时推演前置/后置任务，高亮缺失项。
- **DDL 提醒**：自动识别截止时间并生成提醒调度任务。

---

2. 技术选型

结合“快速响应、高准确率、易降级演示”的项目需求，本系统的技术选型如下：

2.1 后端与开发语言

- **语言**：`Python 3.10+`。拥有最完善的 AI 与文档处理生态。
- **Web 框架**：`FastAPI + Uvicorn`。天然支持异步与类型提示，极适合处理大模型耗时的流式请求、多文件上传与跨域配置。
- **数据校验**：`jsonschema`。针对大模型输出的结构进行严格校验，防止脏数据击穿前端。

2.2 AI 大模型引擎

- **接口标准**：`OpenAI 兼容接口`。系统不绑定单一供应商，支持灵活配置 OneAPI 中转站、

DeepSeek、通义千问等国内外模型，保证网络与额度的可用性。

- **双引擎架构**：采用“大模型语义理解 + 纯代码逻辑约束”双引擎，模型负责“读懂”，代码（如复杂度公式重算、字段互斥裁剪）负责“规训”。

2.3 文档解析与多模态

- **PDF 引擎**：`pymupdf` / `pypdf`。采用双引擎互为兜底，保障文字型 PDF 的稳定提取。
- **图片 OCR**：视觉大模型（如 GPT-4o）结合本地开源 `RapidOCR`。当大模型接口不可用时，自动降级为本地 OCR，确保功能不中断。

2.4 存储与前端交互

- **存储层**：轻量级 JSON 原子写。由于系统定位为会话式信息整理工具，为降低部署成本，采用本地线程锁+临时文件原子写保障状态持久化，无需额外部署数据库。
- **前端渲染**：支持 HTML/CSS/JS 或 Vue/React，基于 `enabled_sections` 实现三栏（输入区、清单区、原文预览区）联动响应式渲染。

---

3. 系统架构设计

系统采用前后端分离的三层结构：

```
浏览器（前端 5173 / 静态托管）
        │  HTTP + JSON / multipart
        ▼
FastAPI 后端（8000）
  ├─ 入口层：CORS、参数校验、统一响应、异常兜底
  ├─ 能力层：文本抽取 / 图片解析 / PDF 提取 / 准备清单 / 查看依据 / 文件检索 / 追问
  ├─ 抽取层：A 的 service.py（Prompt + LLM + Matter 结构 + Schema 校验 + RAG 检索）
  └─ 存储层：backend/data（准备清单、文件片段索引，JSON 原子写）
        │
        ▼
大模型（OpenAI 兼容接口，可配置）
  文本模型：LLM_MODEL          视觉模型：TEAM_LLM_MODEL
  无 Key 时：离线 Mock + 本地 OCR（RapidOCR）
```

核心设计原则：**模型只负责"读懂"，结构、判定与降级由代码兜底**，保证演示可复现、失败有反馈。

3.1 接口清单

接口	作用
`POST /api/analyze`	文字 → Matter
`POST /api/check`	用户勾选 + 自述 → 缺项核对
`POST /api/prepare-list` / `save` / `GET`	准备清单生成 / 保存 / 读取
`POST /api/evidence` 、 `/index`	要求 → 原文依据（无依据标为系统建议）
`POST /api/analyze-image`	截图 → Matter（视觉模型 / 本地 OCR）
`POST /api/analyze-pdf`	文字型 PDF → Matter
`POST /api/files/search`	材料上传、片段编号、检索
`POST /api/ask`	先检索再回答，附出处，材料外拒答
`GET /health`	服务状态与当前模式（mock/real）

所有接口统一返回 `{success, data, error}` ，且始终 HTTP 200，前端只需判断 `success` 。

---

4. 核心算法与机制

4.1 大模型状态推演与动态缺项联动机制

本项目并未采用传统的死板规则做状态联动，而是创新性地引入了**“基于大模型常识推理 + 代码层兜底”**的双引擎状态推演机制。

- **交付动作防误勾**：通过严格的语义识别（如“提交、上传、打包、初审”等关键词），确保即使前置材料已齐备，系统也不会将“交付动作”误标记为完成，必须由用户显式操作。
- **顺序与并行常识倒推**：若用户在清单中完成了后续的“录制视频”步骤，系统利用图依赖（DAG）与常识推理，自动沿链式向前回填，将前置的“提交初审”与“准备材料”一并标记为完成。
- **代码层硬核兜底**：针对大模型可能出现的输出漂移，代码层实现了空标题（ `_fix_step_titles` ）与空待办（ `_fix_todo_actions` ）的强行补全；一旦大模型宕机，纯代码的 `_code_level_fallback` 依然能完成基础的非交付类步骤联动。

4.2 基于 RAG 检索增强的防幻觉与原文溯源机制

为了彻底解决大模型在长文本问答中的“胡编乱造”问题，本项目实现了完整的多阶段 RAG（检索增强生成）流程：

1. **多材料切片与编号（Chunking）**：自研 `NotificationRetriever` 引擎，支持按标点与长度对通知进行无损切片，每个片段自带唯一编号（如 `doc_pdf_1_p1_c1` ）、来源文件名与页码。
2. **混合相关性打分**：结合词频覆盖度、连续字串匹配度（LCS 相似度加分）与实体关键词加权，在多

份材料中精准定位 Top-K 相关原文。

3. 【出处：文件名 - 片段ID】。
4. **无依据拦截与降级兜底**：若本地检索未命中（材料外问题），系统直接拦截并返回“未找到依据”；若大模型 API 突发欠费或宕机，系统自动降级为纯本地检索展示，直接抛出最相关的原文片段与出处，确保服务永不宕机、绝不瞎编。

4.3 输出稳定性与结构化解析

1. `0.2·deadline + 0.1·多提醒 + 0.2·多步骤 + 0.2·材料 + 0.1·审核 + 0.1·依赖 + 0.1·周期 + 0.1·子步骤`，得分 `has_sub_steps/has_review/has_dependencies → complex`、`<0.3 simple`、`0.3–0.6 standard`、`>0.6 complex` 判定。
2. **字段裁剪**：`enforce_mode` 按模式删除互斥字段（simple 不出现 materials/step_list/todo_list 等），保证前端渲染契约稳定。
3. **Schema 校验**：`jsonschema` 校验 Matter 结构，失败直接返回可读错误，不把脏数据交给前端。
4. **三层降级**：真实模型 → 本地 OCR（图片）→ 离线 Mock（无 Key），任一层可用即可演示。

4.4 多模态与多格式文档解析

- **文字型 PDF 提取**：兼容 `pymupdf` 与 `pypdf` 双引擎，自动提取逐页文字并打上页码标签。若检测到提取字数极少，系统会自动触发“疑似扫描件风险”预警。
- **截图与 OCR**：
  - 有团队 Key 时调用 `analyze_notification_from_image`（多模态 + 截图专用规则：表格/人数/日期/材料/金额）。
  - 针对“字迹模糊、表格错位、数字被切断”等视觉歧义，输出 `confirmation_needed` 数组，要求用户二次确认。
  - 无 Key 或视觉调用失败时，自动用 **RapidOCR（onnxruntime）** 本地识别，再走统一抽取管道，确保演示不中断。
- **统一限制端点**：`/api/analyze-image/limits`、`/api/analyze-pdf/limits`，前端提示与后端完全一致，避免页面卡死。

4.5 RAG 与可解释性（工程落地）

- **片段编号**：上传材料按 400 字 / 80 字重叠切分，生成 `file_xxx_c1`、`_c2` …，记录 `char_start/char_end`。
- **检索**：`POST /api/files/search` 对片段做中文二元组重叠打分（命中片段 + 精确子串加权），返回文件名、片段号、原文、分数。
- **追问**：`POST /api/ask` 先检索；有 Key 时把检索片段作为上下文交给 `ask_question`，无 Key 时直接返回最相关原文与出处。
- **不敢答**：检索为空时返回"在已上传的材料中没有找到…无法回答"，不调用模型编造。
- **依据**：`POST /api/evidence` 对材料/步骤/待办/附件/截止/预警/要点返回 `source_ref.quote`；无引用的条目标 `is_suggestion=true`，前端以黄色"系统建议"展示。

---

5. 核心功能实现（后端）

- **统一入口**：`main.py` 负责 CORS、参数校验（空文本 / 超长 10000 字）、multipart 大小与格式校验、统一异常兜底。
- **统一管道**：`analysis_pipeline.analyze_with_fallback` 负责"真实模型 / 离线 Mock"切换，文本、PDF、图片三条入口共用。
- **存储**：准备清单（`data/prepare_lists.json`）与文件索引（`data/files.json`）采用线程锁 + 临时文件原子写，容器通过 volume 持久化。
- **状态联动**：勾选材料 → 关联待办自动 `completed`；依赖全部完成 → `blocked` 解除为 `pending`；状态枚举兼容 `pending/completed/blocked/need_confirm`。

---

6. 测试与部署

6.1 自动化测试（8 套，全部通过）

脚本	项数	覆盖
`smoke_test.py`	10	健康检查、正常/空/超长文本、非法 JSON、CORS、Mock 性能
`smoke_test_check.py`	10	缺项差集、全勾选、自述冲突、参数兜底
`smoke_test_prepare.py`	11	生成、保存、读取、修改内容持久化
`smoke_test_evidence.py`	8	索引、单条依据、无依据 → 建议、参数校验
`smoke_test_image.py`	6	limits、合法图、格式/大小/空文件
`smoke_test_pdf.py`	6	文字型 PDF、扫描件、页数/大小限制
`smoke_test_files.py`	10	上传、片段编号、检索、追问、材料外拒答
`smoke_test_acceptance.py`	3 样本	三档模式端到端验收

6.2 部署方案

- **Docker**：`backend/Dockerfile`（含 pypdf、RapidOCR）+ `docker-compose.yml`，`backend/data` volume 持久化，内置 HEALTHCHECK。
- **Nginx**：反向代理 + HTTPS，配置 `proxy_read_timeout 180s`、`client_max_body_size 12m`。
- **详细文档**：详见 `docs/部署说明.md`。